UNIT-1

IoT TECHNOLOGIES

Introduction to IoT- Introduction, Physical Design of IOT, Logical design of IoT, IoT enabling Technologies, IoT Levels & Development Templates, Difference between IoT and M2M, SDN and NFV for IoT, Need for IoT systems management, Simple Network Management Protocol (SNMP), Network operator requirements.

1.1 INTRODUCTION

 The Internet of Things represents the whole way from collecting data, processing it, taking an action corresponding to the signification of this data to storing everything in the cloud. All this is made possible by the internet

 The Internet of things has become a very widely spread concept in the last few years. The reason for this is mainly the need to computerize and control most of the surrounding objects and have access to data in real time.

 Example: Parking sensors, about phones which can check the weather and so on.

The Internet of Things (IoT) refers to a network of interconnected physical objects such as devices, machines, vehicles, or people embedded with sensors, software, and unique identifiers that enable them to collect, exchange, and process data over a network without requiring direct human-to-human or human-to-computer interaction.

1.1.1 Definition & Characteristics of IoT Definition:

A dynamic global n/w infrastructure with self-configuring capabilities based on standard and interoperable communication protocols where physical and virtual ―things have identities, physical attributes and virtual personalities and use intelligent interfaces, and are seamlessly integrated into information n/w, often communicate data associated with users and their environments.

1.1.2 Characteristics of IoT

i)**Dynamic & Self Adapting**:

IoT devices and systems may have the capability to dynamically adapt with the changing contexts and take actions based on their operating conditions, user‘s context or sensed environment.

Eg: The surveillance system comprising of a number of surveillance cameras. The surveillance camera can adapt modes based on whether it is day or night. The surveillance system is adapting itself based on context and changing conditions.

ii)**Self Configuring:**

IOT devices have self-configuring capability, allowing a large number of devices to work together to provide certain functionality. These devices have the ability configure themselves setup networking, and fetch latest software upgrades with minimal manual or user interaction.

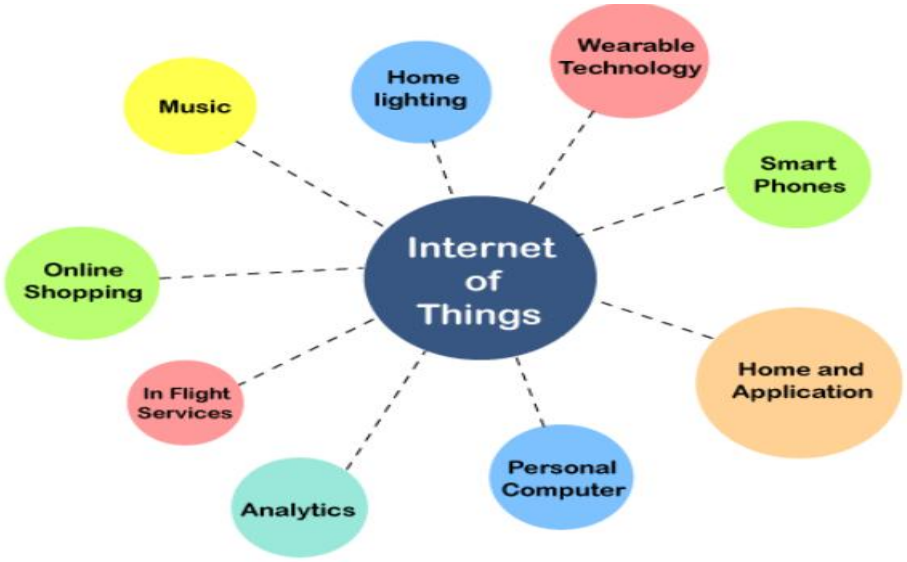
iii) **Inter Operable Communication Protocols:** support a number of interoperable communication protocols and can communicate with other devices and also with infrastructure.

iv) **Unique Identity:** Each IoT device has a unique identity and a unique identifier(IP address).

v) **Integrated into Information Network:** that allow them to communicate and exchange data with other devices and systems.

1.1.3 Applications of IoT:

1) Home
2) Cities
3) Environment
4) Energy
5) Retail
6) Logistics
7) Agriculture
8) Industry
9) Health &LifeStyle



1.2 PHYSICAL DESIGN OF IOT :

The "Things" in IoT usually refers to IoT devices which have unique identities and can perform remote sensing, actuating and monitoring capabilities.

IoT devices can:

- ✓ Exchange data with other connected devices and applications (directly or indirectly), or
- ✓ Collect data from other devices and process the data locally or
- ✓ Send the data to centralized servers or cloud-based application back-ends for processing the data,
- ✓ Perform some tasks locally and other tasks within the IoT infrastructure, based on temporal and space constraints.
- ✓ The physical design of IoT refers to the actual hardware components and real-world objects involved in an IoT system.
- ✓ It focuses on how devices are built, connected, and interact physically with the environment.
- ✓ Physical design knowledge is crucial for selecting suitable devices and sensors, ensuring seamless integration, and optimizing connectivity options in IoT systems.
- ✓ It enables power efficient strategies, facilitates edge computing, and

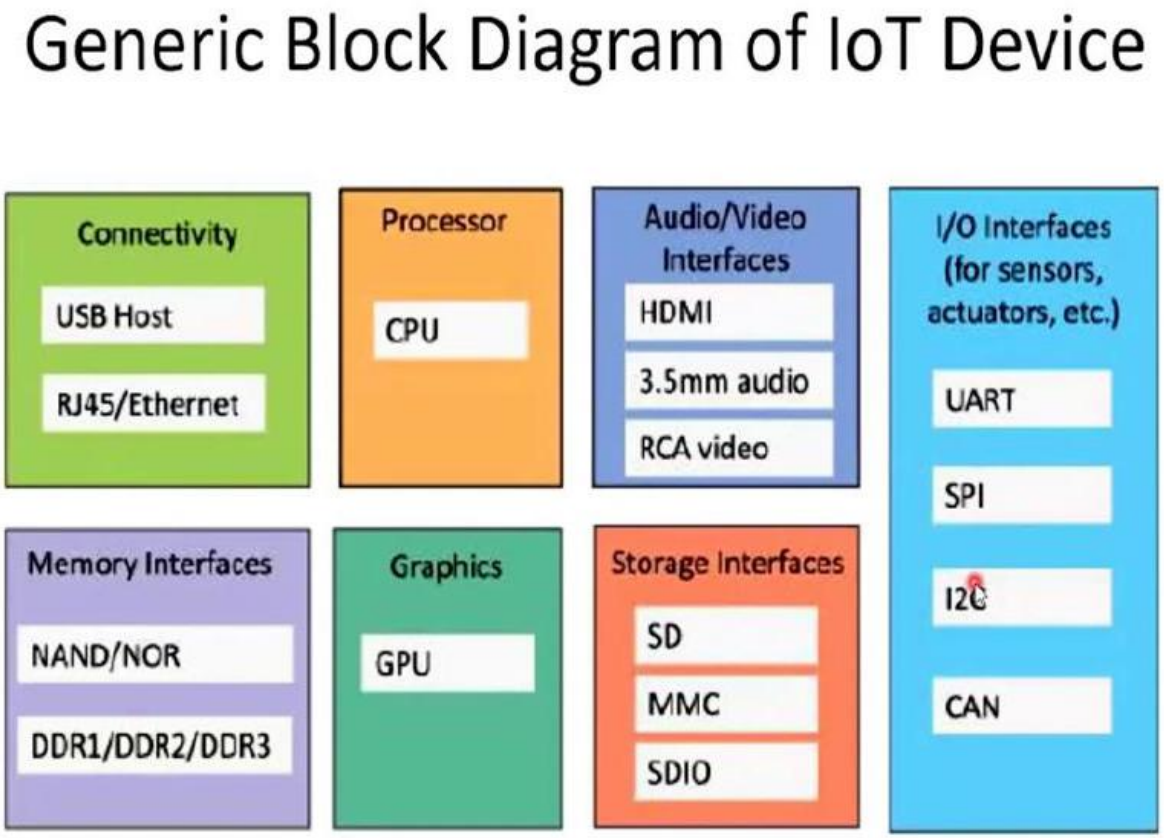
enhances reliability and resilience through redundancy and failover mechanisms. This knowledge ensures robust, efficient, and reliable IoT ecosystems.

1.2.1 Generic block diagram of an IoT Device

• An IoT device may consist of several interfaces for connections to other devices, both wired and wireless.

• I/O interfaces for sensors

• Interfaces for Internet connectivity



•

Memory and storage interfaces

• Audio/video interfaces.

• HDMI: High definition multimedia Interface.

• 3.5mm: Audio Jack which headphone adapter.

• RCA: Radio corporation of America.

• UART: Universal Asynchronous Receiver Transmitter.

• SPI: Serial Peripheral Interface.

• I2C: Inter integrated circuit

• CAN: Controller Area Network used for Micro-controllers and devices to communicate.

• SD: Secure digital (memory card)

• MMC: multimedia card

• SDIO: Secure digital Input Output

• GPU: Graphics processing unit.

• DDR: Double data rate

1.2.2 Components of Physical Design of IoT:

1. **Devices:**

    - Things are physical objects connected to the internet that can sense, communicate, and perform actions.
      Each device has a unique identity and can interact with other devices or systems.
    - They form the foundation of any IoT system by representing real-world objects.

    Example

- Smart bulb
- Smart watch

2. **Sensors:**

    - Sensors are hardware components that detect physical or environmental conditions such as temperature, light, motion, or humidity.
    - They convert real-world signals into digital data that can be processed by IoT systems.
    - Sensors enable data collection from the physical environment.

Example

- Temperature sensor
- Motion sensor

3. **Actuators:**

    - Actuators are devices that convert electrical signals into physical movement or action.
    - They perform actions based on commands received from the controller or cloud.

    Example

- LED
- Motor

4. **Embedded Systems / Microcontrollers**:

    - Embedded systems are small computing units designed to perform specific control functions in IoT devices.
    - They process sensor data, execute programmed logic, and control actuators.

    Example

- Arduino
- Raspberry Pi

5. **Communication Modules:**

    Communication modules allow IoT devices to connect and exchange data over a network.
    They support technologies such as Wi-Fi, Bluetooth, ZigBee, and cellular networks.
    These modules enable remote monitoring and control of devices.

Example

- Wi-Fi module
- Bluetooth module

6. **Communication Protocols:**
    - Communication protocols define the rules for data transmission between IoT devices and servers.
    - They ensure reliable, secure, and efficient data exchange.

    Example

- MQTT
- HTTP

7. **Power Supply / Power Source:**
    - The power supply provides the necessary energy for IoT devices to function.
    - It can be provided through batteries, electrical power, or renewable sources like solar energy.
    - Efficient power management is critical for long-term IoT operation.

    Example

- Battery
- Solar panel

1.2.3 IoT Protocols in Physical Design

Protocols define how devices talk across different layers.

- Link Layer: Ethernet (802.3), Wi-Fi (802.11).
- Network Layer: IPv4, IPv6.
- Transport Layer: TCP, UDP.
- Application Layer: HTTP, MQTT (lightweight messaging for IoT).

1.2.3a IoT Protocols in Each layer:

a) Link Layer:

Protocols determine how data is physically sent over the network‘s physical layer or medium. Local network connect to which host is attached. Hosts on the same link exchange data packets over the link layer using link layer protocols. Link layer determines how packets are coded and signalled by the h/w device over the medium to which the host is attached.

Protocols:

- ✓ 802.3-Ethernet: IEEE802.3 is collection of wired Ethernet standards for the link layer. Eg: 802.3 uses co-axial cable; 802.3i uses copper twisted pair connection; 802.3j uses fiber optic connection; 802.3ae uses Ethernet overfiber.
- ✓ 802.11-WiFi: IEEE802.11 is a collection of wireless LAN(WLAN) communication standards including extensive description of link layer. Eg: 802.11a operates in 5GHz band, 802.11b and 802.11g operates in 2.4GHz band, 802.11n operates in 2.4/5GHz band, 802.11ac operates in 5GHz band, 802.11ad operates in 60Ghzband.
- ✓ 802.16 -WiMax: IEEE802.16 is a collection of wireless broadband standards including exclusive description of link layer. WiMax provide data rates from 1.5 Mb/s to 1Gb/s.
- ✓ 802.15.4-LR-WPAN: IEEE802.15.4 is a collection of standards for low rate wireless personal area network(LR-WPAN). Basis for high level communication protocols such as ZigBee. Provides data rate from 40kb/s to250kb/s.
- ✓ 2G/3G/4G-Mobile Communication: Data rates from 9.6kb/s(2G) to up to100Mb/s(4G).

b) Network/Internet Layer:

Responsible for sending IP datagrams from source n/w to destination n/w. Performs the host addressing and packet routing. Datagrams contains source and destination address.

Protocols:

- ✓ IPv4: Internet Protocol version4 is used to identify the devices on a n/w using a hierarchical addressing scheme. 32 bit address. Allows total of 2*32addresses.
- ✓ IPv6: Internet Protocol version6 uses 128 bit address scheme and allows 2*128 addresses.
- ✓ 6LOWPAN:(IPv6 over Low power Wireless Personal Area Network) operates in 2.4 GHz frequency range and data transfer 250 kb/s.

c) Transport Layer:

Provides end-to-end message transfer capability independent of the underlying n/w. Set up on connection with ACK as in TCP and without ACK as in UDP. Provides functions such as error control, segmentation, flow control and congestion control.

Protocols:

- ✓ TCP: Transmission Control Protocol used by web browsers(along with HTTP and HTTPS), email(along with SMTP, FTP). Connection oriented and stateless protocol. IP Protocol deals with sending packets, TCP ensures reliable transmission of protocols in order. Avoids n/w congestion and congestioncollapse.
- ✓ UDP: User Datagram Protocol is connectionless protocol. Useful in time sensitive applications, very small data units to exchange. Transaction oriented and stateless protocol. Does not provide guaranteed delivery.

d) Application Layer:

Defines how the applications interface with lower layer protocols to send data over the n/w. Enables process-to-process communication using ports.

Protocols:

- ✓ HTTP: Hyper Text Transfer Protocol that forms foundation of WWW. Follow request response model Stateless protocol.
- ✓ CoAP: Constrained Application Protocol for machine-to-machine(M2M) applications with constrained devices, constrained environment and constrained n/w. Uses client-server architecture.
- ✓ WebSocket: allows full duplex communication over a single socket connection.
- ✓ MQTT: Message Queue Telemetry Transport is light weight messaging protocol based on publish-subscribe model. Uses client server architecture. Well suited for constrained environment.
- ✓ XMPP: Extensible Message and Presence Protocol for real time communication and streaming XML data between network entities. Support client-server and server-server communication.
- ✓ DDS: Data Distribution Service is data centric middleware standards for device-to-device or machine-to-machine communication. Uses publish-subscribe model.
- ✓ AMQP: Advanced Message Queuing Protocol is open application layer protocol for business messaging. Supports both point-to-point and publish-subscribe model.

1.3 LOGICAL DESIGN OF IoT:

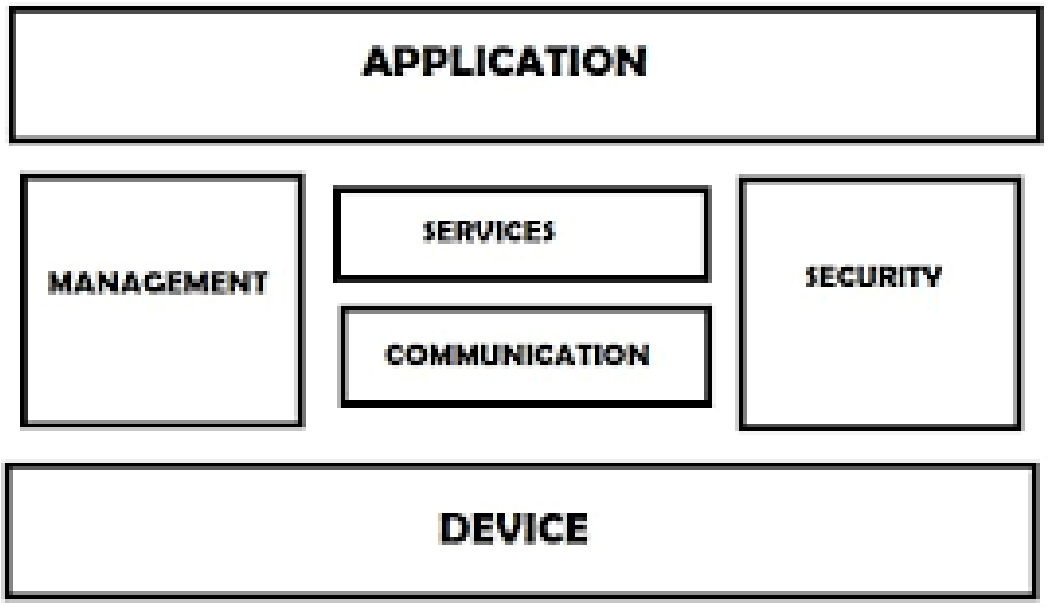
Refers to an abstract represent of entities and processes without going into the low level specifies of implementation.

1) IoT Functional Blocks
2) IoT Communication Models
3) IoT Comm. APIs

1.3.1 IoT Functional Blocks:

Provide the system the capabilities for identification, sensing, actuation, communication and management

- ✓ Device: An IoT system comprises of devices that provide sensing, actuation, monitoring and control functions.
- ✓ Communication: handles the communication for IoT system.
- ✓ Services: for device monitoring, device control services, data publishing services and services for device discovery.
- ✓ Management: Provides various functions to govern the IoT system.
- ✓ Security: Secures IoT system and priority functions such as authentication, authorization, message and context integrity and data security.
- ✓ Application: IoT application provide an interface that the users can use to control and monitor various aspects of IoT system.



1.3.2 IoT Communication Models:

A) Request-Response

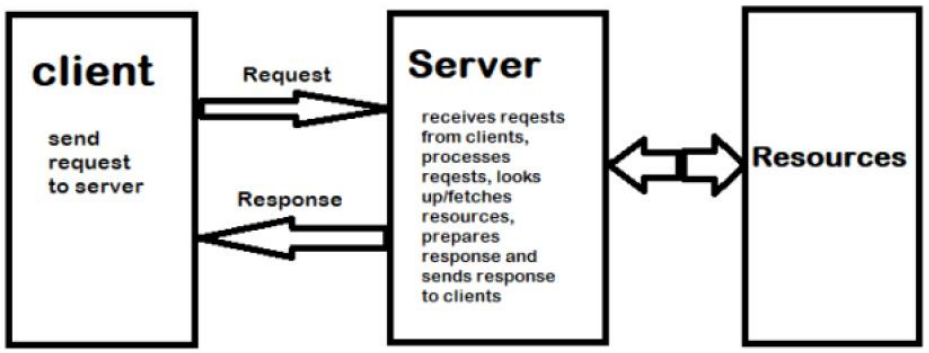
B) Publish-Subscribe

C)Push-Pull

D) Exclusive Pair

1.3.2A) Request-Response

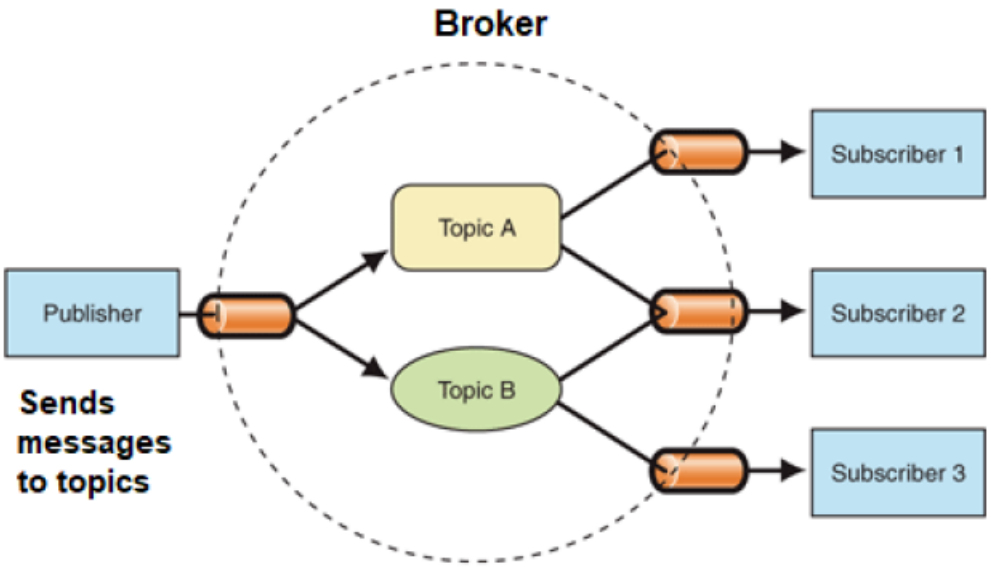
Request-Response is a communication model in which the client sends requests to the server and the server responds to the requests. When the server receives a request, it decides how to respond, fetches the data, retrieves resource representations, prepares the response, and then sends the response to the client.



Request-Response Communication Model

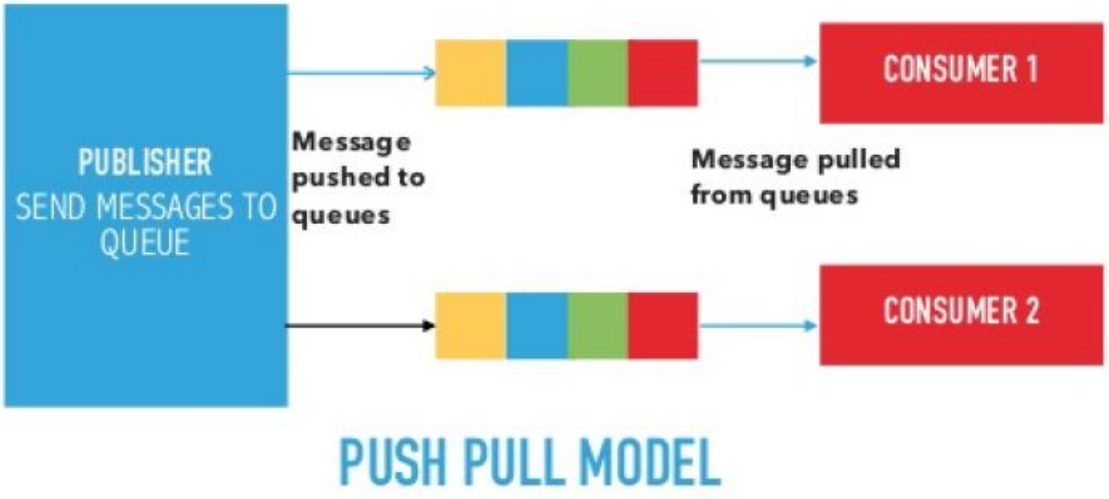
1.3.2B) Publish-Subscribe communication model:

- Publish-Subscribe is a communication model that involves publishers, brokers and consumers.
- Publishers are the source of data. Publishers send the data to the topics which are managed by the broker. Publishers are not aware of the consumers.
- Consumers subscribe to the topics which are managed by the broker.
- When the broker receives data for a topic from the publisher, it sends the data to all the subscribed consumers



1.3.2C) Push-Pull communication model:

- Push-Pull is a communication model in which the data producers push the data to queues and the consumers pull the data from the queues. Producers do not need to be aware of the consumers.
- Queues help in decoupling the messaging between the producers and consumers.
- Queues also act as a buffer which helps in situations when there is a mismatch between the rate at which the producers push data and the rate at which the consumers pull.



1.3.2D) Exclusive Pair communication model:

- Exclusive Pair is a bidirectional, fully duplex communication model that uses a persistent connection between the client and server.
- Once the connection is setup it remains open until the client sends a request to close the connection.
- Client and server can send messages to each other after connection setup.

EXCLUSIVE PAIR COMMUNICATION MODEL

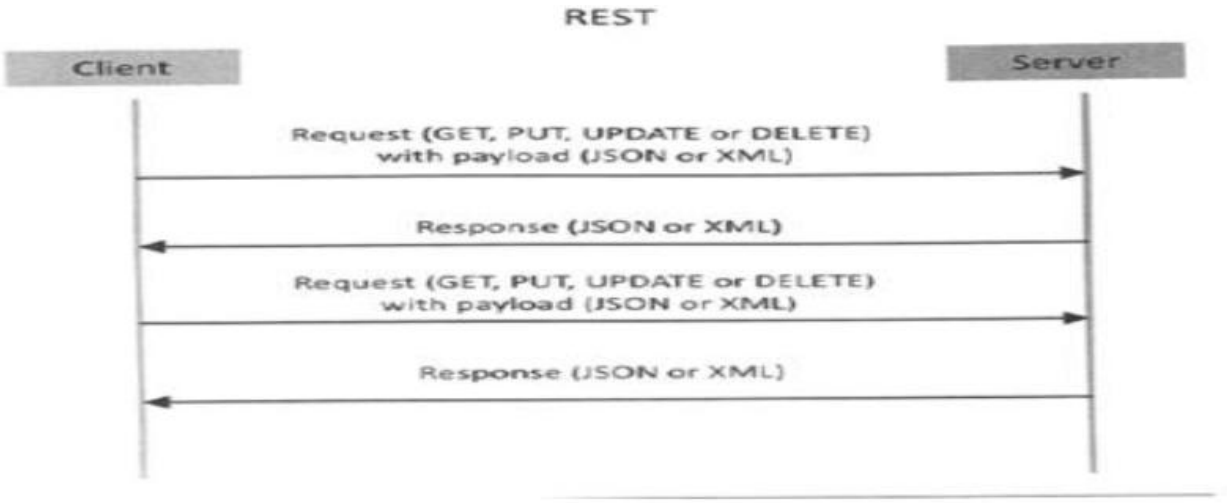
1.3.3 IoT Communication APIs:

a) REST based communication APIs(Request-Response Based Model)

b) WebSocket based Communication APIs(Exclusive PairBasedModel)

1.3.3A) Request-Response model used by REST:

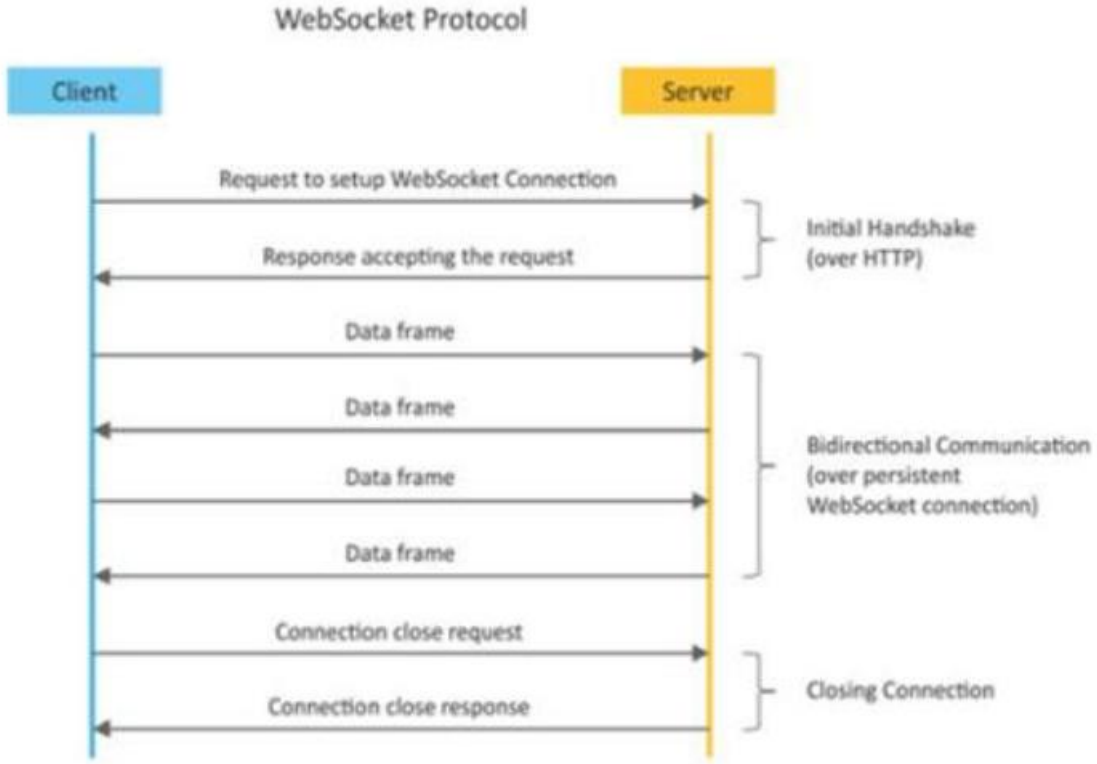
RESTful webservice is a collection of resources which are represented by URIs. RESTful web API has a base URI(e.g: http://example.com/api/tasks/). The clients and requests to these URIs using the methods defined by the HTTP protocol(e.g: GET, PUT, POST or DELETE). A RESTful web service can support various internet media types.



Request-response model used by REST

1.3.3B) WebSocket Based Communication APIs:

WebSocket APIs allow bi-directional, full duplex communication between clients and servers. WebSocket APIs follow the exclusive pair communication model.

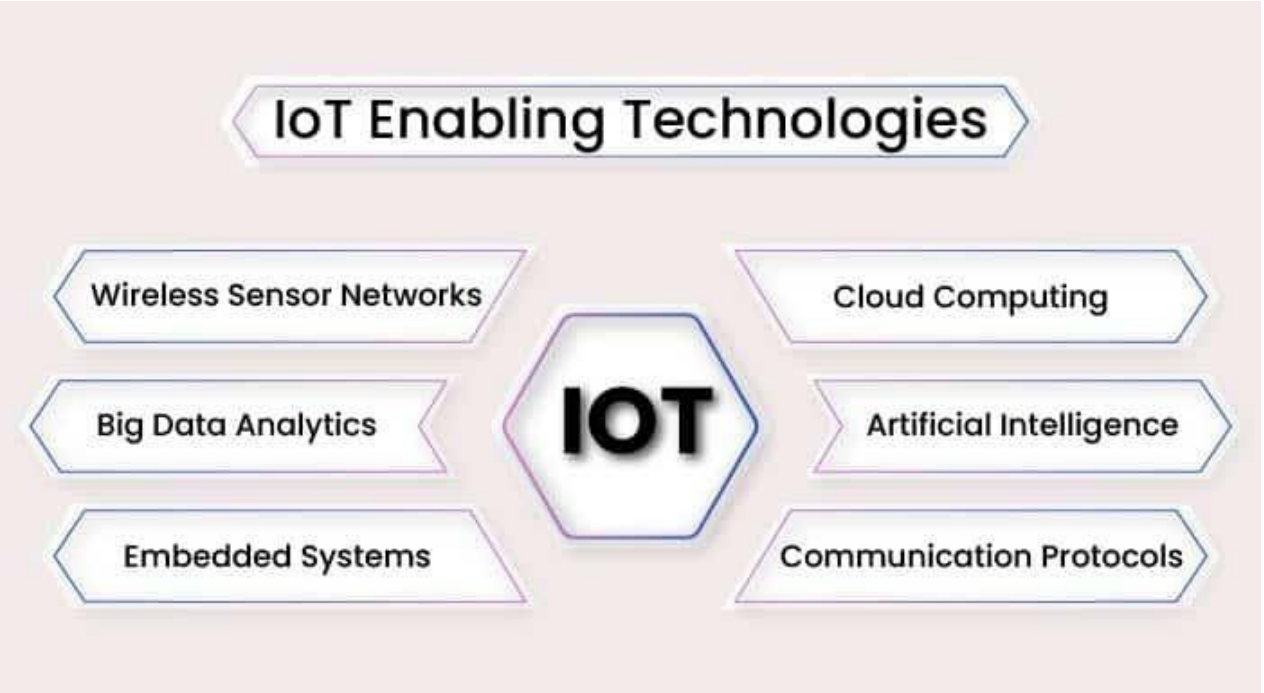


1.4 IOT ENABLING TECHNOLOGIES

- The logical design of IoT describes the software-level architecture and functional components of an IoT system.
- It explains how data flows**,** how components interact**,** and how services are provided.
- Logical design focuses on what the system does, not the physical hardware.

IoT enabling technologies:

1. Wireless Sensor Network
2. Cloud Computing
3. Big Data Analytics
4. Communications Protocols
5. Embedded System



1.4.1 Wireless Sensor Networks

A wireless sensor network comprises of distributed devices with sensors which are used to monitor the environmental and physical conditions. A WSN consist of a number of end nodes and routers and a co-ordinator. The coordinator collects the data from all the nodes. Coordinator also acts as a gateway that connects the WSN to the internet.

WSNs used in IoT systems are described as follows:

- ✓ Weather Monitoring System: in which nodes collect temp, humidity and other data, which is aggregated and analyzed.
- ✓ Indoor air quality monitoring systems: to collect data on the indoor air quality and concentration of various gases.
- ✓ Soil Moisture Monitoring Systems: to monitor soil moisture at various locations.
- ✓ Surveillance Systems: use WSNs for collecting surveillance data(motion data detection).
- ✓ Smart Grids : use WSNs for monitoring grids at various points.

- ✓ Structural Health Monitoring Systems: Use WSNs to monitor the health of structures(building, bridges) by collecting vibrations from sensor nodes deployed at various points in the structure.

WSNs are enabled by wireless communication protocols such as IEEE 802.15.4. Zig Bee is one of the most popular wireless technologies used by WSNs .Zig Bee specifications are based on IEEE 802.15.4. Zig Bee operates 2.4 GHz frequency and offers data rates upto 250 KB/s and range from 10 to 100meters.

1.4.2 Cloud Computing

Cloud computing is a transformative computing paradigm that involves delivering applications and services over the internet. Cloud computing involves provisioning of computing, networking and storage resources on demand and providing these resources as metered services to the users, in a "pay as you go". Cloud computing resources can be provisioned on-demand by the users, without requiring interactions with the cloud service provider. The process of provisioning resources is automated. Cloud computing services are offered to users in different forms.

- ✓ ***Infrastructure-as-a-service(IaaS):***Provides users the ability to provision computing and storage resources. These resources are provided to the users as a virtual machine instances and virtual storage.
- ✓ ***Platform-as-a-Service(PaaS):*** Provides users the ability to develop and deploy application in cloud using the development tools, APIs, software libraries and services provided by the cloud service provider.
- ✓ ***Software-as-a-Service(SaaS):*** Provides the user a complete software application or the user interface to the application itself. The cloud service provider manages the underlying cloud infrastructure including servers, network, operating systems, storage, and application software.

1.4.3 Big data Analysis

Big data is defined as collections of data sets whose volume , velocity or variety is so large that it is difficult to store, manage, process and analyze the data using traditional databases and data processing tools.

Some examples of big data generated by IoT are Sensor data generated by IoT systems.

- ✓ Machine sensor data collected from sensors established in industrial and energy systems.
- ✓ Health and fitness data generated IoT devices.
- ✓ Data generated by IoT systems for location and tracking vehicles.
- ✓ Data generated by retail inventory monitoring systems.

The underlying characteristics of Big Data are

- ✓ ***Volume:*** There is no fixed threshold for the volume of data for big data. Big data is used for massive scale data.
- ✓ ***Velocity:*** Velocity is another important characteristics of Big Data and the primary reason for exponential growth of data.
- ✓ ***Variety:*** Variety refers to the form of data. Big data comes in different forms such as structured or unstructured data including test data, image , audio, video and sensor data .

1.4.4Communication Protocols:

Communication Protocols form the back-bone of IoT systems and enable network connectivity and coupling to applications.

- ✓ Allow devices to exchange data over network.
- ✓ Define the exchange formats, data encoding addressing schemes for device and routing of packets from source to destination.
- ✓ It includes sequence control, flow control and retransmission of lost packets.

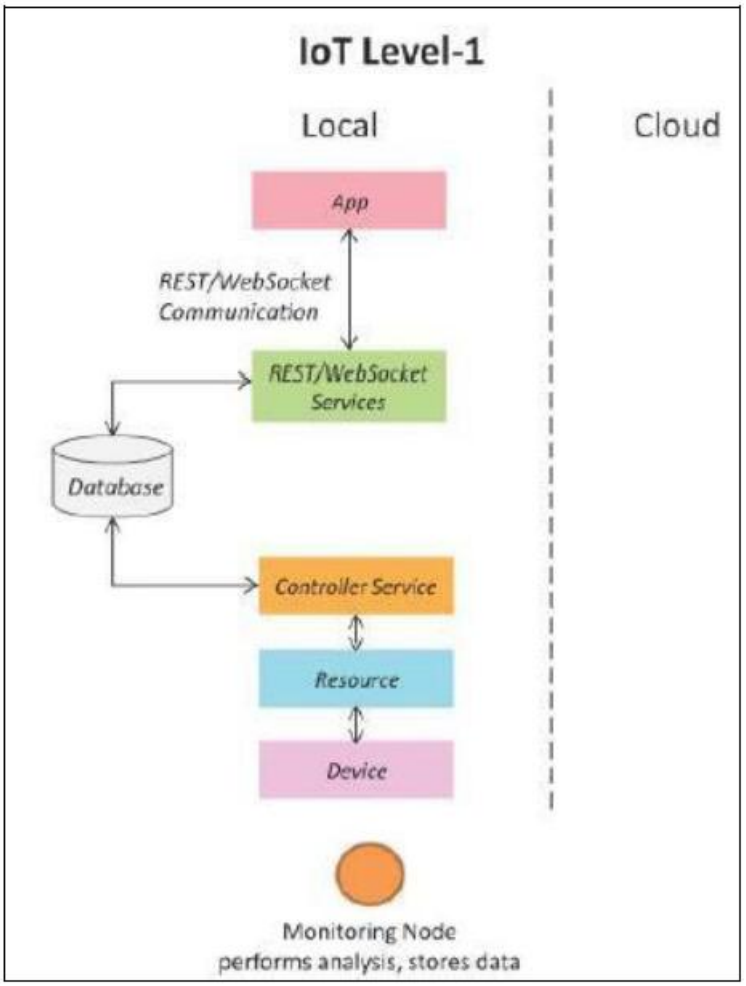
1.4.5 Embedded Systems:

Embedded Systems is a computer system that has computer hardware and software embedded to perform specific tasks. Key components of embedded system include microprocessor or micro controller, memory (RAM, ROM, Cache), networking units (Ethernet Wi-Fi Adaptor), input/output units (Display, Keyboard, etc..,) and storage (Flash memory). Embedded System range from low cost miniaturized devices such as digital watches to devices such as digital cameras, POS terminals, vending machines, appliances etc.,

1.5 IOT LEVELS AND DEPLOYMENT TEMPLATES

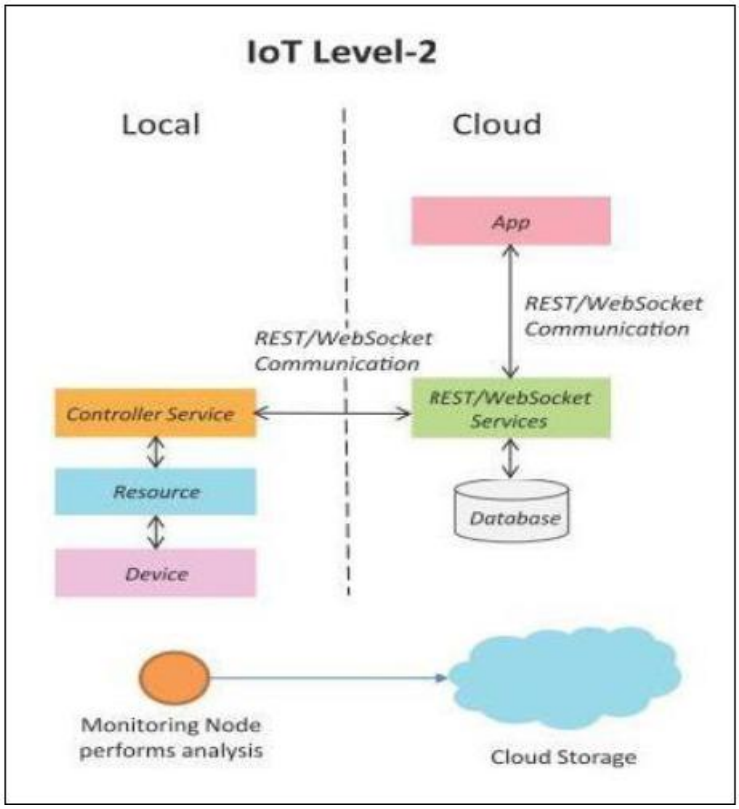
1.5.1 IoT Level-1

Level-1 IoT systems has a single node that performs sensing and/or actuation, stores data, performs analysis and host the application. Suitable for modelling low cost and low complexity solutions where the data involved is not big and analysis requirement are not computationally intensive. An e.g., of IoT Level1 is Home automation. The system consist of a single node that allows controlling the lights and appliances in a home the device used in this system interfaces with the lights and appliances using electronic rely switches. The status information of each light or appliances is maintained in a local database. REST services deployed locally allow retrieving and updating the state of each lighter appliance in the status database. The controller service continuously monitors the state of each light or appliance by retrieving the light from the database.



1.5.2 IoT Level 2

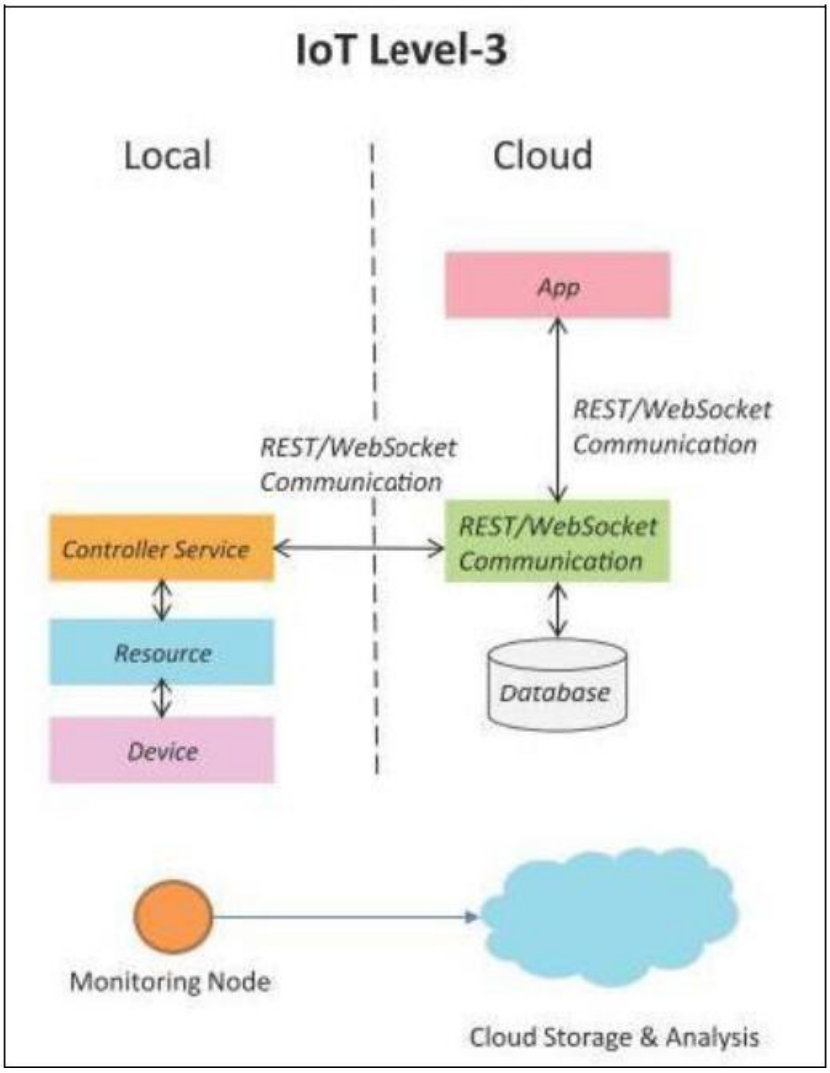
IoT Level2 has a single node that performs sensing and/or actuating and local analysis as shown in fig. Data is stored in cloud and application is usually cloud based. Level2 IoT systems are suitable for solutions where data are involved is big, however, the primary analysis requirement is not computationally intensive and can be done locally itself. An e,g., of Level2 IoT system for Smart Irrigation. The system consists of a single node that monitors the soil moisture level and controls the irrigation system. The device used system collects soil moisture data from sensors. The controller service continuously monitors the moisture level. A cloud based REST web service is used for storing and retrieving moisture data which is stored in a cloud database. A cloud based application is used for visualizing the moisture level over a period of time which can help in making decision about irrigation schedule.



1.5.3 IoT Level 3

This System has a single node. Data is stored and analyzed in the cloud application is cloud based as shown in fig. Level3 IoT systems are suitable for solutions where the data involved is big and analysis requirements are computationally intensive.

The system consists of a single node that monitors the vibration levels for the package being shipped . The device in this system uses accelerometer and gyroscope sensor for monitoring vibration levels. The controller serves in the sensor data to the cloud in a real time using a websocket service. The data is stored in the cloud and also visualizing the cloud based applications . The analysis components in the cloud can trigger alerts if the vibration level becomes greater than the threshold.



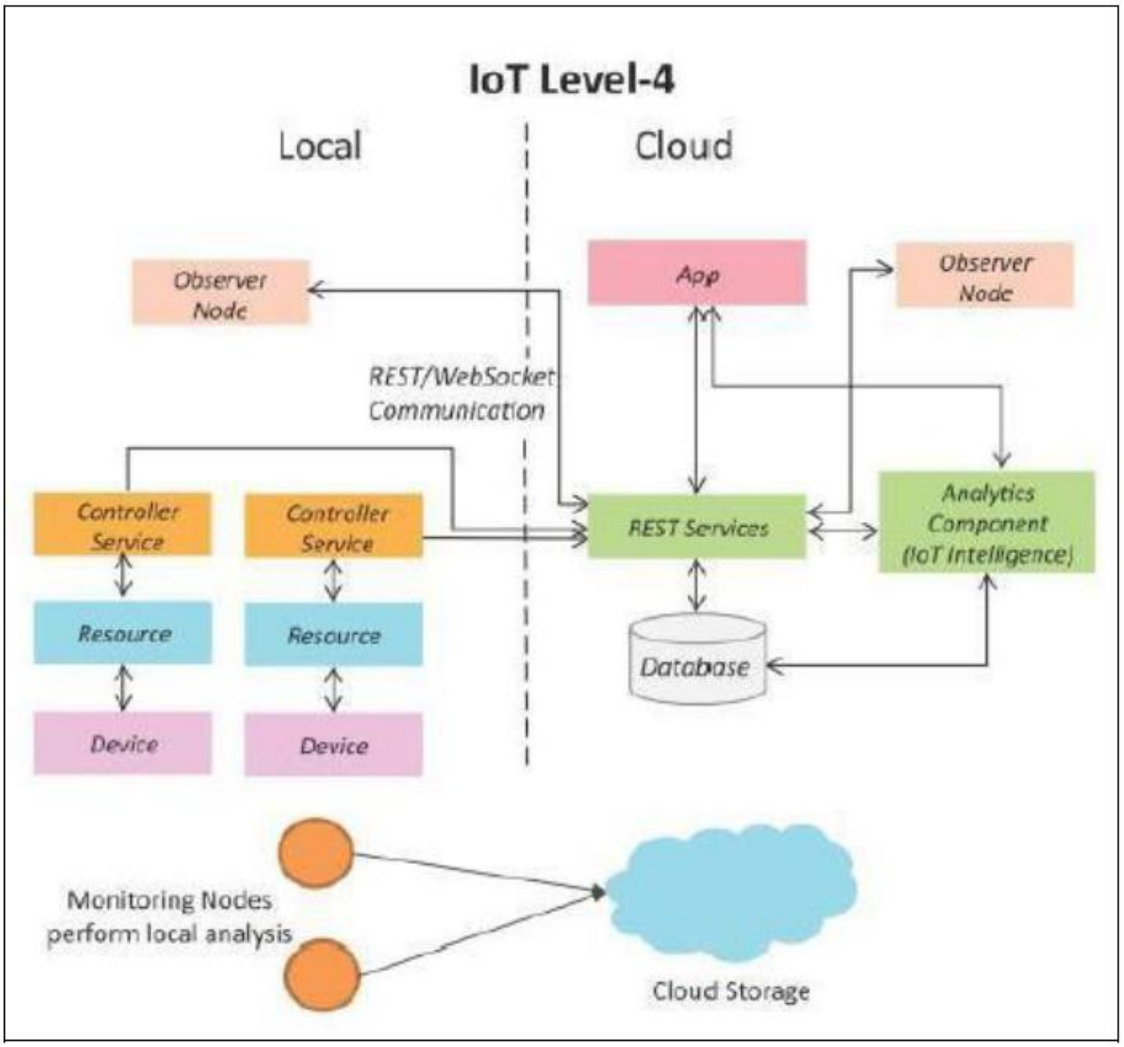
1.5.4 IoT Level 4

This System has multiple nodes that perform local analysis. Data is stored in the cloud and application is cloud based as shown in fig. Level4 contains local and cloud based observer nodes which can subscribe to and receive information collected in the cloud from IoT devices. Level 4 IoT systems are suitable for solutions where multiple nodes are required, the data involved in big and the analysis requirements are computationally intensive.

Example : IoT System for Noise Monitoring.

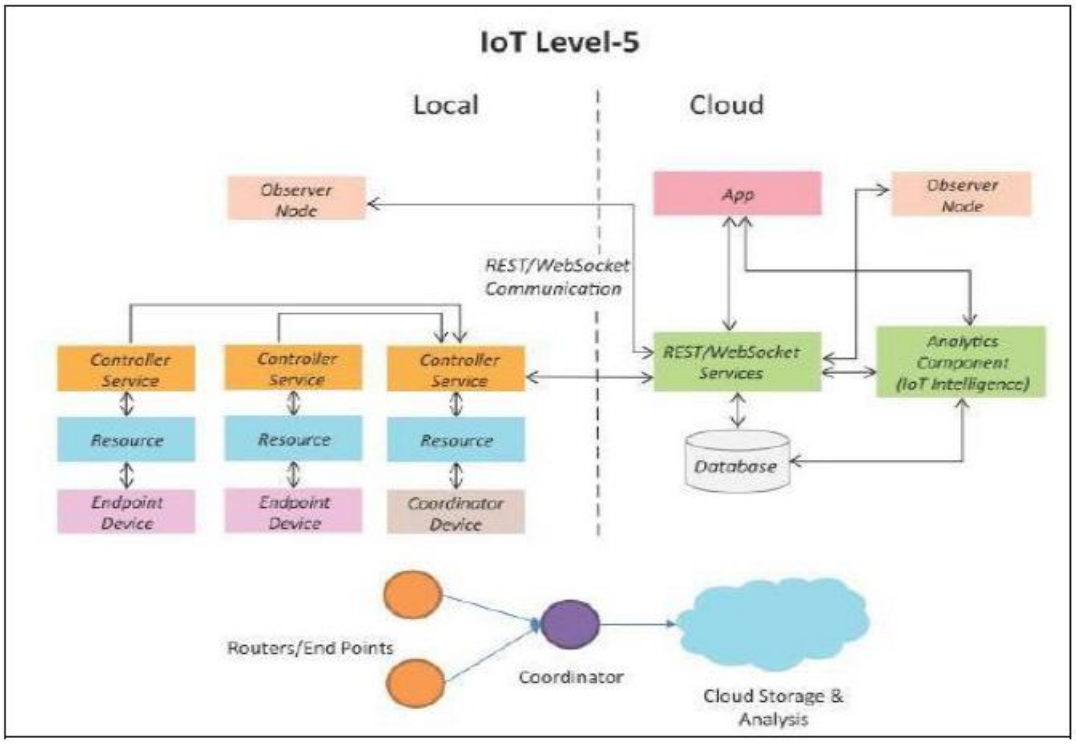
The system consists of multiple nodes placed in different locations for monitoring noise levels in an area. The nodes in this example are equipped with sound sensors. Nodes are independent of each other. Each nodes runs its owner controller service that sends the data to the cloud . The data is stored in cloud database. The analysis of

data collected from a number of nodes is done in the cloud. A cloud based application is used for visualizing the aggregated data.



1.5.5 IoT Level 5

System has multiple end nodes and one coordinator node as shown in fig. The end nodes that perform sensing and/or actuation. Coordinator node collects data from the end nodes and sends to the cloud. Data is stored and analyzed in the cloud and application is cloud based. Level5 IoT systems are suitable for solution based on wireless sensor network, in which data are high intensive.



Example :IoT system for Forest Fire Detection.

The system consists of multiple nodes placed in different locations for monitoring temperature, humidity and CO2 levels in a forest. The end nodes in this example are equipped with various sensors such as temperature, humidity and CO2. The coordinator node collects the data from the end nodes and act as a gateway that provides internet connectivity to the IoT system. The controller service on the coordinator device sends the collected data to the cloud. The data is stores in a cloud database. The analysis of data is done in the computing cloud to aggregate the data and make predictions. A cloud based applications is used for visualizing the data .

1.5.6 IoT Level 6.

System has multiple independent end nodes that perform sensing and/or actuation and sensed data to the cloud. Data is stored in the cloud and application is cloud based as shown in fig. The analytics component analyses the data and stores the result in the cloud data base. The results are visualized with the cloud based applications. The centralized controller is aware of the status of all endnodes and sends control commands to the nodes.

Example weather monitoring system

The system consists of multiple nodes placed in different locations for monitoring temperatures,humidity and pressure in an area.the end nodes are equipped with various sensors(such as temperature,humidity and pressure).the end nodes send the data to the cloud realtime using a websocketservice.the data is stored in a cloud database. The analysis of data is done in a cloud to aggregate a data and make predictions.acloud based application is used for visualizing the data.

1.6 Difference between IoT and M2M

S.No	Feature	IoT (Internet of Things)	M2M (Machine to Machine)
1	**Definition**	Devices connected via internet to exchange data and provide smart services	Direct communication between machines/devices without internet dependency
2	**Connectivity**	Internet-based; uses cloud, Wi-Fi, cellular, LPWAN	Can be wired or wireless; often doesn't use internet
3	**Scope**	Broad: integrates devices, cloud, analytics, AI, applications	Narrow: focused on point-to- point communication
4	**Data Processing**	Usually sent to cloud for analytics and decision making	Often processed locally or used directly by machines
5	**Human Involvement**	Can interact with humans via apps, dashboards	Typically no human interaction; autonomous
6	**Network Complexity**	Complex; needs scalability, security, protocol management	Simple; fewer devices and simpler protocols
7	**Examples**	Smart homes, wearable devices, smart cities, connected cars	Industrial sensors, ATMs, vending machines

8	**Flexibility**	Highly flexible; can integrate multiple platforms and services	Limited flexibility; works mainly in closed environments
9	**Standardization**	Uses standardized internet protocols (HTTP, MQTT, CoAP)	Often proprietary or simpler communication protocols
10	**Business Focus**	Consumer-centric and industrial applications	Mostly industrial and operational use cases

M2M versus the IoT

M2M	IoT
M2M is about direct communication between machines.	The IoT is about sensors automation and Internet platform.
It supports point-to-point communication.	It supports cloud communication.
Devices do not necessarily rely on an Internet connection.	Devices rely on an Internet connection.
M2M is mostly hardware-based technology.	The IoT is both hardware- and software-based technology.
Machines normally communicate with a single machine at a time.	Many users can access at one time over the Internet.
A device can be connected through mobile or other network.	Data delivery depends on the Internet protocol (IP) network.

1.7 SDN vs NFV for IoT:

Software defined Networking(SDN)

SDN stands for Software Defined Network which is a networking architecture approach. It enables the control and management of the network using software applications. Through Software Defined Network (SDN) networking behavior of the entire network and its devices are programmed in a centrally controlled manner through software applications using open APIs.

To understand software-defined networks, we need to understand the various planes involved in networking.

1. Data Plane

2. Control Plane

Data plane:

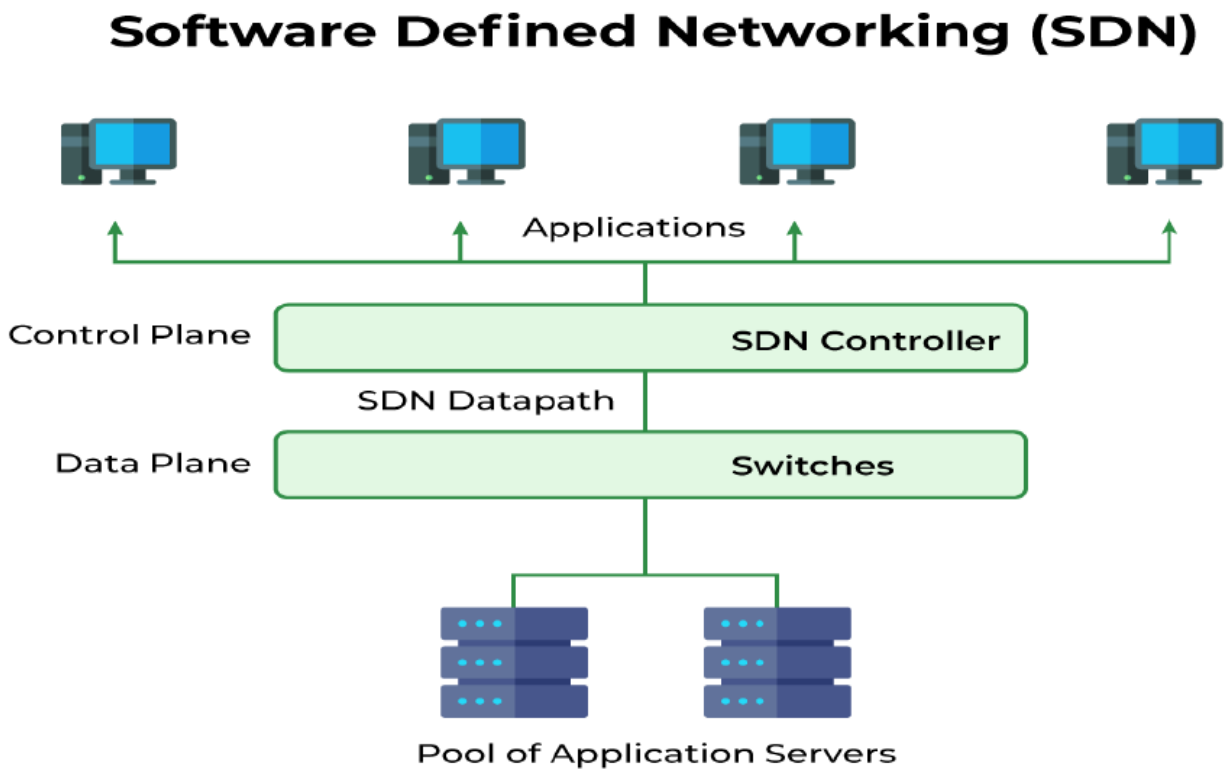
All the activities involving as well as resulting from data packets sent by the end-user belong to this plane. This includes:

- Forwarding of packets.
- Segmentation and reassembly of data.
- Replication of packets for multicasting.

Control plane:

All activities necessary to perform data plane activities but do not involve end-user data packets belong to this plane. In other words, this is the brain of the network. The activities of the control plane include:

- Making routing tables.
- Setting packet handling policies.



1.7A)Why SDN is Important?

- ✓ **Better Network Connectivity:** SDN provides very better network connectivity for sales, services, and internal communications. SDN also helps in faster data sharing.
- ✓ **Better Deployment of Applications:** Deployment of new applications, services, and many business models can be speed up using Software Defined Networking.
- ✓ **Better Security:** Software-defined network provides better visibility throughout the network. Operators can create separate zones for devices that require different levels of security. SDN networks give more freedom to operators.
- ✓ **Better Control with High Speed:** Software-defined networking provides better speed than other networking types by applying an open standard software-based controller.

In short, it can be said that- SDN acts as a "Bigger Umbrella or a HUB" where the rest of other networking technologies come and sit under that umbrella and get merged with another platform to bring out the best of the best outcome by decreasing the traffic rate and by increasing the efficiency of data flow.

1.7B) Where is SDN Used?

- ✓ Enterprises use SDN, the most widely used method for application deployment, to deploy applications faster while lowering overall deployment and operating costs. SDN allows IT administrators to manage and provision network services from a single location.
- ✓ Cloud networking software-defined uses white-box systems. Cloud providers often use generic hardware so that the Cloud data center can be changed and the cost of CAPEX and OPEX saved.

1.7C) Components of Software Defining Networking (SDN)

The three main components that make the SDN are:

1. **SDN Applications:** SDN Applications relay requests or networks through SDN Controller using API.
2. **SDN controller:** SDN Controller collects network information from hardware and sends this information to applications.
3. **SDN networking devices:** SDN Network devices help in forwarding and data processing tasks.

1.7D) SDN Architecture

In a traditional network, each switch has its own data plane as well as the control plane. The control plane of various switches exchange topology information and hence construct a forwarding table that decides where an incoming data packet has to be forwarded via the data plane.

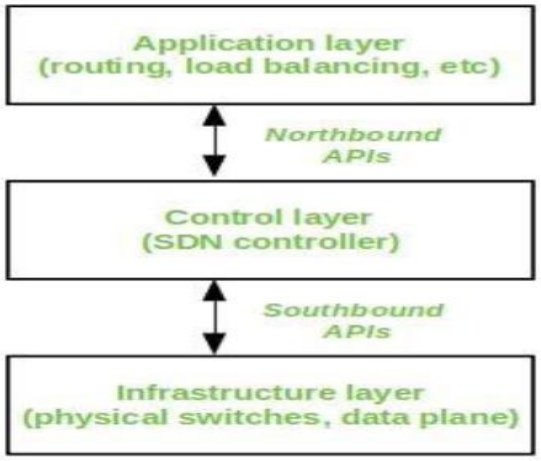
Software-defined networking (SDN) is an approach via which we take the control plane away from the switch and assign it to a centralized unit called the SDN controller. Hence, a network administrator can shape traffic via a centralized console without having to touch the individual switches.

The data plane still resides in the switch and when a packet enters a switch, its forwarding activity is decided based on the entries of flow tables, which are pre-assigned by the controller. A flow table consists of match fields (like input port number and packet header) and instructions. The packet is first matched against the match fields of the flow table entries. Then the instructions of the corresponding flow entry are executed. The instructions can be forwarding the packet via one or multiple ports, dropping the packet, or adding headers to the packet. If a packet doesn't find a corresponding match in the flow table, the switch queries the controller which sends a new flow entry to the switch. The switch forwards or drops the packet based on this flow entry.

A typical SDN architecture consists of three layers.

- **Application layer:** It contains the typical network applications like intrusion detection, firewall, and load balancing
- **Control layer:** It consists of the SDN controller which acts as the brain of the network. It also allows hardware abstraction to the applications written on top of it.
- **Infrastructure layer:** This consists of physical switches which form the data plane and carries out the actual movement of data packets.

The layers communicate via a set of interfaces called the north-bound APIs(between the application and control layer) and southbound APIs(between the control and infrastructure layer).



SDN Architecture

Advantages of SDN

- ✓ The network is programmable and hence can easily be modified via the controller rather than individual switches.
- ✓ Switch hardware becomes cheaper since each switch only needs a data plane.
- ✓ Hardware is abstracted, hence applications can be written on top of the controller independent of the switch vendor.

- ✓ Provides better security since the controller can monitor traffic and deploy security policies. For example, if the controller detects suspicious activity in network traffic, it can reroute or drop the packets.

Disadvantages of SDN

- ✓ The central dependency of the network means a single point of failure, i.e. if the controller gets corrupted, the entire network will be affected.
- ✓ The use of SDN on large scale is not properly defined and explored.

1.8 Network FunctionsVirtualization

The term "Network Functions Virtualization" (NFV) refers to the use of virtual machines in place of physical network appliances. There is a requirement for a hypervisor to operate networking software and procedures like load balancing and routing by virtual computers. A network functions virtualization standard was first proposed at the OpenFlow World Congress in 2012 by the European Telecommunications Standards Institute (ETSI), a group of service providers that includes AT&T, China Mobile, BT Group, Deutsche Telekom, and many more.

1.8.1 Need of NFV:

With the help of NFV, it becomes possible to separate communication services from specialized hardware like routers and firewalls. This eliminates the need for buying new hardware and network operations can offer new services on demand. With this, it is possible to deploy network components in a matter of hours as opposed to months as with conventional networking. Furthermore, the virtualized services can run on less expensive generic servers.

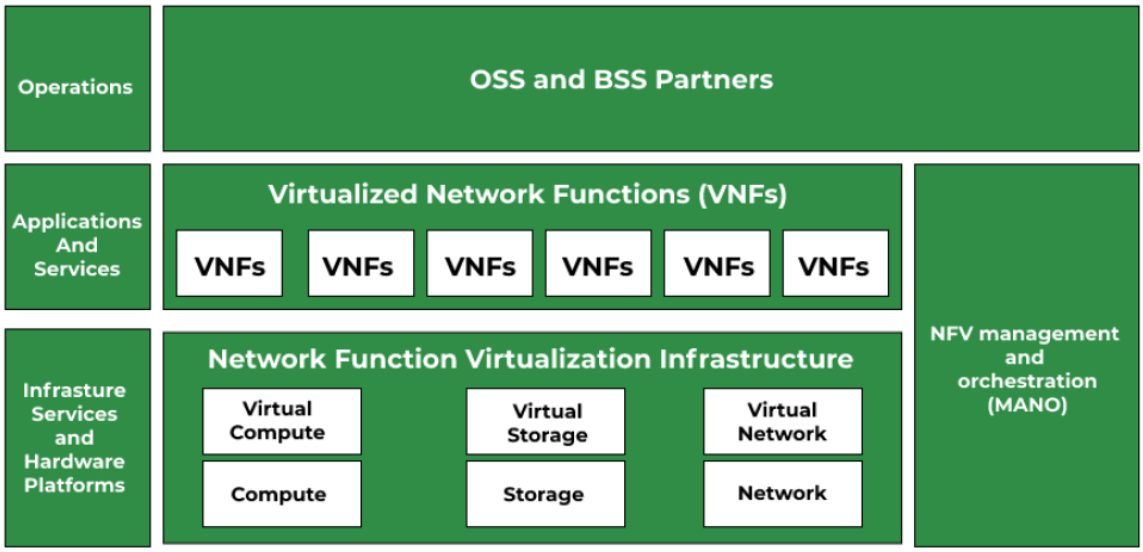
1.8.2 Advantages:

- ✓ Lower expenses as it follows Pay as you go which implies companies only pay for what they require.
- ✓ Less equipment as it works on virtual machines rather than actual machines which leads to fewer appliances, which lowers operating expenses as well.

- ✓ Scalability of network architecture is quite quick and simple using virtual functions in NFV. As a result, it does not call for the purchase of more hardware.

1.8.3 Working:

Usage of software by virtual machines enables to carry out the same networking tasks as conventional hardware. The software handles the task of load balancing, routing, and firewall security. Network engineers can automate the provisioning of the virtual network and program all of its various components using a hypervisor or software-defined networking controller.



1.8.4 Benefits of NFV:

✓Many service providers believe that advantages outweigh the issues of NFV.

✓Traditional hardware-based networks are time-consuming as these require network administrators to buy specialized hardware units, manually configure them, then join them to form a network. For this skilled or well-equipped worker is required.

✓It costs less as it works under the management of a hypervisor, which is significantly less expensive than buying specialized hardware that serves the same purpose.

✓Easy to configure and administer the network because of a virtualized network. As a result, network capabilities may be updated or added instantly.

1.8.5 Risks of NFV:

Security hazards do exist, though, and network functions virtualization security issues have shown to be a barrier to widespread adoption among telecom companies. The following are some dangers associated with implementing network function virtualization that service providers should take into account:

- **Physical security measures do not work:** Comparing virtualized network components to locked-down physical equipment in a data center enhances their susceptibility to new types of assaults.
- Malware is difficult to isolate and contain: Malware travels more easily among virtual components running on the same virtual computer than between hardware components that can be isolated or physically separated.
- **Network activity is less visible:** Because traditional traffic monitoring tools struggle to detect potentially malicious anomalies in network traffic going east-west between virtual machines, NFV necessitates more fine-grained security solutions.

1.8.6 NFV Architecture:

An individual proprietary hardware component, such as a router, switch, gateway, firewall, load balancer, or intrusion detection system, performs a specific networking function in a typical network architecture. A virtualized network substitutes software programs that operate on virtual machines for these pieces of hardware to carry out networking operations.

Three components make up an NFV architecture:

 Centralized virtual network infrastructure: The foundation of an NFV infrastructure can be either a platform for managing containers or a hypervisor that abstracts the resources for computation, storage, and networking.

 Applications: Software delivers many forms of network functionality by substituting for the hardware elements of a conventional network design (virtualized network functions).

 Framework: To manage the infrastructure and provide network functionality, a framework is required (commonly abbreviated as MANO, meaning Management, Automation, and Network Orchestration).

S.No	Feature	SDN (Software-Defined Networking)	NFV (Network Function Virtualization)
1	**Definition**	Separates control plane and data plane for centralized network management	Virtualizes network functions (firewalls, routers) to run on general-purpose hardware
2	**Purpose**	Network programmability and flexibility	Reduce dependency on dedicated hardware, cost efficiency
3	**Control**	Centralized control over network traffic	Distributed control via virtualized functions
4	**Use in IoT**	Manages massive IoT traffic efficiently	Quick deployment of network services for IoT devices
5	**Scalability**	High; can handle millions of IoT devices	High; virtual functions can scale up/down on demand
6	**Latency**	Helps optimize routing to reduce latency	Can reduce latency if functions are deployed near IoT edge
7	**Cost**	Requires some specialized SDN controllers	Reduces hardware costs by using standard servers
8	**Flexibility**	Can change network paths and policies dynamically	Network functions can be deployed, modified, or removed quickly
9	**Security**	Can implement security policies centrally	Security functions like firewalls can be virtualized and deployed as needed

10	**Examples**	Dynamic routing, IoT traffic load balancing	Virtual routers, firewalls, load balancers for IoT networks

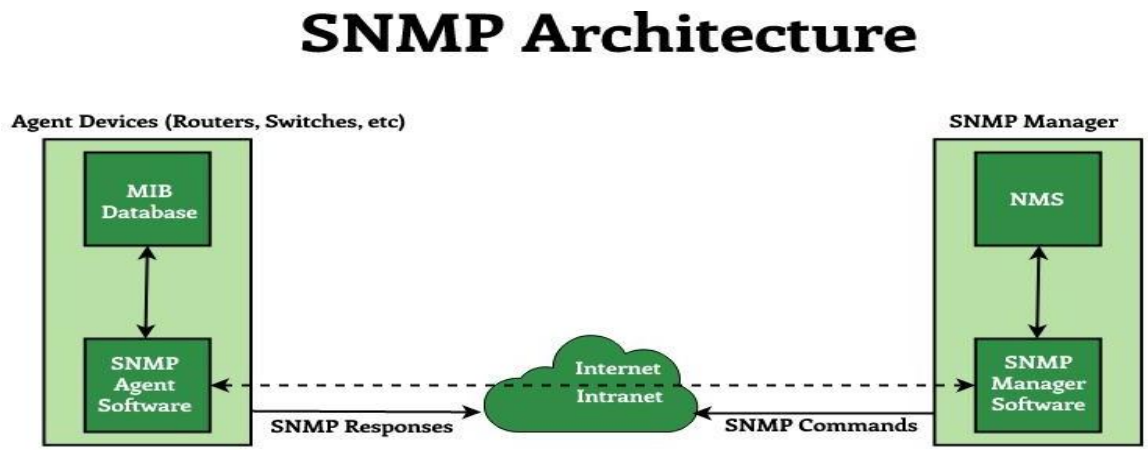
1.9 Need for IoT Systems Management:

1. **Device Monitoring:** Continuously track the status, health, and performance of IoT devices to prevent failures.
2. **Configuration Management:** Ensure all devices are properly configured and aligned with the system requirements.
3. **Software/Firmware Updates:** Remotely update devices to fix bugs, add features, or patch security vulnerabilities.
4. **Security Management:** Protect devices and data from unauthorized access, malware, and cyberattacks.
5. **Data Management:** Efficiently collect, store, and process the massive amounts of data generated by IoT devices.
6. **Fault Detection & Recovery:** Quickly detect device failures or network issues and restore normal operations.
7. **Scalability Management:** Easily add, remove, or reconfigure devices and services as the IoT network grows.
8. **Energy/Resource Optimization:** Monitor and optimize battery life, bandwidth, and other resources of IoT devices.
9. **Compliance & Standards:** Ensure devices and networks adhere to industry regulations and standards.
10. **Performance Optimization:** Monitor network traffic, latency, and device performance to improve overall system efficiency.

1.10 Simple Network Management Protocol (SNMP):

Simple Network Management Protocol (SNMP) is a standard protocol used to

monitor, manage, and control network devices such as routers, switches, servers, printers, and IoT devices. It allows network administrators to collect information about devices, detect network problems, and configure network parameters remotely. SNMP works over the TCP/IP protocol suite and provides a centralized way to manage large networks efficiently. It uses a simple structure of data objects to represent device information and enables automatic alerts and reporting for issues in real time. SNMP is widely used because it is lightweight, scalable, and supports heterogeneous devices across different manufacturers.



How SNMP Works

1. **Managed Devices:** Network devices (routers, switches, sensors, IoT devices) that store information about their status.
2. **SNMP Agent:** Software installed on managed devices that collects and sends device information to the manager.**Network Management System (NMS):** Central system used by administrators to monitor, manage, and control all SNMP-enabled devices.
3. **MIB (Management Information Base):** A virtual database on each device containing information about its status, performance, and configuration.
4. **Operations:**

   - ✓ **Get:** NMS requests information from the agent.

- ✓ **Set:** NMS changes a configuration on the agent.
- ✓ **Trap:** Agent sends an unsolicited alert to the NMS if a problem occurs.

Example of SNMP

- A **network administrator** wants to monitor a company's router bandwidth usage.
- The **SNMP agent** on the router tracks bandwidth, uptime, and errors.
- The **NMS** queries the router using SNMP and displays the data on a dashboard.
- If the router goes down, the agent sends a **trap alert** to notify the administrator.

Applications of SNMP

1. **Network Monitoring:** Track the health, performance, and status of all network devices.
2. **Fault Management:** Detect, diagnose, and respond to network problems automatically.
3. **Configuration Management:** Remotely configure devices and update network settings.
4. **Performance Management:** Monitor traffic, bandwidth usage, CPU load, and memory usage.
5. **Security Management:** Monitor unauthorized access attempts and ensure compliance with security policies.
6. **IoT Device Management:** Manage large-scale IoT networks by monitoring sensors, actuators, and gateways.
7. **Data Center Management:** Monitor servers, storage devices, and network devices in real time.

8. **Alerting and Reporting:** Generate automatic alerts (traps) for critical events and prepare periodic performance reports.

1.11 Network Operator Requirements:

Network Operator Requirements refer to the set of needs and conditions that network operators must fulfill to ensure efficient, secure, and reliable operation of communication networks. These requirements include maintaining network performance, monitoring traffic, managing devices, ensuring security, and supporting scalability. Meeting these requirements helps operators provide continuous service, optimize resources, and adapt to emerging technologies like IoT, 5G, and cloud networks. Essentially, these requirements guide how a network should be designed, monitored, and managed for optimal functionality.

Example

A telecom company managing a city-wide 5G network must ensure all base stations and routers are functioning correctly.

- Operators monitor network traffic to prevent congestion during peak hours.
- If a router fails, they must detect it quickly, fix it, or reroute traffic automatically to maintain service.
- Security measures like firewalls and encryption are applied to prevent unauthorized access.

In **IoT (Internet of Things)**, network operators must support a very large number of heterogeneous devices while ensuring reliable, secure, and scalable communication.

1. Connectivity Requirements

- Support for **massive number of devices** (mMTC)
- Ability to handle **low-power and low-data-rate** communications
- Support for multiple access technologies
  (NB-IoT, LTE-M, 5G, Wi-Fi, LPWAN, LoRaWAN, Zigbee)
- **Seamless coverage**, including remote and indoor areas

2. Scalability Requirements

- Network must scale to **millions of IoT nodes**
- Efficient **device onboarding and management**
- Support for dynamic addition and removal of devices

3. Performance Requirements

- **Low latency** for real-time IoT applications (healthcare, automation)
- **High reliability** for mission-critical systems
- Optimized handling of **small and intermittent data packets**
- QoS differentiation for different IoT services

4. Energy Efficiency Requirements

- Support for **low-power operation**
- Features like **sleep modes**, power-saving signaling
- Extended **battery life** (years in some IoT devices)

5. Security Requirements

- Secure **device authentication and authorization**
- **End-to-end data encryption**
- Protection against IoT-specific threats (botnets, spoofing)
- Secure **firmware updates** and device lifecycle management

6. Network Management Requirements

- Centralized **IoT device management**
- Remote monitoring and diagnostics
- Fault detection and self-healing mechanisms
- Data collection and analytics support

7. Data Handling Requirements

- Efficient **data aggregation and filtering**

- Support for **edge computing** to reduce core network load
- Integration with **cloud platforms**

8. Business & Regulatory Requirements

- Cost-effective solutions for low-revenue devices
- Flexible **billing models** (per device, per data usage)
- Compliance with **data privacy and telecom regulations**

9. Interoperability Requirements

- Compatibility with different **IoT platforms and vendors**
- Support for standard IoT protocols
  (MQTT, CoAP, HTTP, AMQP)

Applications

1. **Telecommunication Networks:** Ensuring continuous connectivity for mobile, internet, and voice services.
2. **Data Centers:** Managing servers, storage, and network devices efficiently.
3. **IoT Networks:** Monitoring and controlling large-scale IoT deployments like smart cities, smart homes, and industrial IoT.
4. **Enterprise Networks:** Ensuring corporate networks are secure, high-performing, and scalable for employees.
5. **Cloud Networks:** Managing virtualized resources and services for cloud providers to maintain performance and reliability.
6. **Traffic and Bandwidth Management:** Optimizing network resources to prevent congestion and improve user experience.
7. **Security & Compliance:** Protecting sensitive data, monitoring for attacks, and ensuring adherence to regulatory standards.